Symbolic Execution is (Not Quite) All You Need

Sophie Smithburg

***Abstract*—We present a technique for extracting verified dispatch automata from ELF binaries via symbolic execution and partition refinement. The extracted labeled transition system is bisimilar to the concrete dispatch behavior, proved in Lean 4 with no admitted axioms. Guards and updates are expressed in the quantifier-free bitvector fragment of SMT-LIB, enabling automated vulnerability detection via Datalog queries. In a blind trial, a 119-line Soufflé program correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375 in mjs v1.20.1 from anonymized dispatch facts alone.**

***Index Terms*—symbolic execution, formal verification, labeled transition systems, partition refinement, LangSec, Lean 4, Datalog**

I. Introduction

No one writes specs; everyone writes implementations. But you can't prove things about implementations directly—you can about specs. So in order to have an effective ratchet in the sense of Metzger's LangSec keynote [1]—addressing the specification-implementation gap [2]—we want to be able to get specs from implementations.

We can model a programming language executing on a host machine as a host labeled transition system H [3] which, after having been fed $\mathcal{I}$, the implementation, simulates G—the PL is G, its implementation is $\mathcal{I}$, and the host machine is H. To prove a recovered G' faithfully captures G, we would need G in a formalism that shares the same proof basis as G'—but in practice, this is never the case. So what we do instead is prove that G' simulates $H_{\mathcal{I}}$, by way of projecting away dispatch-irrelevant state and transitions.

We have implemented a technique and have an early proof of concept of a system that, by way of symbolic execution and Paige-Tarjan partition refinement [4], can extract a strongly bisimilar symbolic labeled transition system from an ELF binary—guards and updates expressed in the quantifier-free bitvector fragment of SMT-LIB. We have successfully used this to generate vulnerability detection queries in Soufflé [5], a Datalog dialect oriented towards static analysis. In a blind trial, Claude Opus 4.6 was given only two toy synthetic examples of a vulnerability class, wrote a 119-line Soufflé program that distinguished bounded from unbounded recursion, and when applied blind to anonymized facts from mjs v1.20.1, correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375. We also used the technique to confirm the absence of certain terminal emulator vulnerability classes in libtsm, by passing a description of the vulnerability classes along with the extracted LTS to an LLM agent.

This paper and its Lean 4 mechanization were developed with extensive use of Claude Code and ChatGPT, some use of Codex CLI, and a smattering of other LLM tools in earlier prototypes.

II. Background

The Language-Theoretic Security thesis [2] holds that computational power is a dimension of the attack surface: an implementation that exceeds the computational requirements of its specification should be considered broken. When an implementation admits more computational power than its specification requires, the excess constitutes a weird machine—an unintended computational substrate available to an attacker.

We introduce the *dispatch automaton*: the control-flow automaton (CFA) [6] of a program's dispatch loop, projected to retain only dispatch-relevant state and with dispatch-irrelevant transitions elided. A parser's dispatch automaton is the grammar it implements. An interpreter's dispatch automaton is its opcode dispatch table. A terminal emulator's dispatch automaton is its VT state machine.

The dispatch automaton's computational class determines what the implementation can do beyond its specification. We adopt the standard hierarchy from automata theory: finite languages (fixed command dispatch), regular (flat input processing), visibly pushdown [7] (recursive descent with bracketed call/return), and context-sensitive or higher (accumulated state influencing dispatch). The classification boundary that matters for LangSec is: if a regular-spec parser has a context-sensitive dispatch automaton, the excess computational power is a potential weird machine.

For systems designed for human use, the dispatch automaton is almost always regular or visibly pushdown—humans design in finite terms. The implementation is a finite specification with engineering noise. Our tool recovers the signal.

III. What We Extract

The output of our pipeline is a finite labeled transition system (LTS). We conjecture that this LTS is strongly bisimilar [8] to the concrete dispatch behavior of the input binary, in the sense of van Glabbeek's spectrum [9]. The argument: the extracted LTS contains no internal (τ) transitions—all computation between dispatch points is composed into direct labeled transitions during enumeration, and in the absence of τ-transitions, bisimulation and strong bisimulation coincide. Formalizing this in Lean requires one additional lemma (that deterministic τ-elimination preserves bisimulation), which is in progress.

We note that the extracted LTS may be slightly larger than the pure dispatch specification: stack frame manipulation and other host-level implementation detail are preserved in transition effects, not hidden. We are bisimulating the guest-level system plus a bit of the host-level system at the same time. This may be addressable with better ABI awareness in

a future version; for a work-in-progress report, a slightly too-large LTS with a bit of extra detail is acceptable.

Guards and updates are expressed in the quantifier-free bitvector fragment of SMT-LIB (QF_UFBV), with uninterpreted functions for memory load and store operations. Expressions are simplified via CVC5 [10] and deduplicated into a shared symbol DAG, reducing LTS files from 45 MB to 7 KB in representative cases.

Our algorithm converges by way of ensuring that we have reached a fixed point of composing the dispatch body with itself. But that can only happen if there is finite state in the underlying automaton. We get bisimulation precisely when the system is visibly pushdown or regular or finite in its dispatch automaton—because the precise criterion that pushes a system from visibly pushdown to counting or Turing is when dispatch can inspect more than the top of a stack, and that is precisely the condition where the closure cannot close. If there is infinite state, there can be infinite distinctions, and our algorithm runs to its iteration bound—you get a portion of the equivalence class set modeled, not missing branches, but having not yet folded all the logic into the fused LTS. If our technique unexpectedly diverges, you have found a weird machine. If it converges, you may have still found a weird machine—but if so, it is a fully specified weird machine, and you have a bisimulation of it. The proof is mechanized in Lean 4 with zero `sorry` and zero admitted axioms.

IV. Method

We take an ELF binary and use Pyvex [11] to lift it to VEX IR. After that point, everything is in Lean with a pretty decent amount of proofs around things. Our work is heavily based on the compositional symbolic execution semantics by Voogd, Laarman, and Lööw [12]; we have ported some of their lemmas to Lean and used their abstraction as our core data structure. A *symbolic branch*—a substitution paired with a path condition guard, following Voogd et al.'s formulation—is the atom of our analysis.

We symbolically execute functions along a weak topological ordering of the Bourdoncle [13] variety. This means that we symbolically execute leaf functions before the functions that call them, and so we can use the symbolic summaries of the callees when we get to the call sites. We have what we call the *dispatch body*, which is basically just a pile of all the symbolic branches from all the functions under analysis—roughly one big switch statement.

We start at the leaves. When we process a leaf, we look at what is on the frontier from that node outward, and we keep checking to see if it composes—is the next symbolic branch reachable from the one we are at? If it is, we compose the two, and if not, we discard the irrelevant branch. At each step we compute the *PC-signature* of the resulting state: a dictionary that maps each guard in the dispatch closure to its three-valued evaluation (true, false, or unknown) in the current abstract state. This signature is the key for our equivalence classes—two states with the same signature behave identically with respect to which symbolic branches they can take next. We stop iteration when a full pass through composing the dispatch body with itself ceases to yield new signatures. This is the fixpoint of the accumulation of PC-signatures.

At various points we do various simplifications: we strip provably non-aliasing stores in a region-aware way, which really helped with the tractability of the export in terms of size. We also use CVC5 [10] to simplify our guards and update expressions in the emitted LTS, and we reference-count any recurring expressions and turn them into a DAG of references for subexpressions.

Our application of Paige-Tarjan partition refinement [4] starts with the PC-signature classes. We split by transition structure: if a transition from one class leads to different target classes for different members, we split. It is an $O(m \log n)$ algorithm, so plenty performant. In practice, the PC-signature classes almost always already are the bisimulation quotient—the refinement step rarely needs to split further.

Pyvex is a key trust boundary for us [14]; it feels like a sensible enough thing to rely on. A collaborator is working on a PDB variant of our system—the goal was always to be underlying IR agnostic, we have just not built for multiple at once yet. We require non-position-independent ELF binaries at present; it may be possible to expand to position-independent executables, the only issue being that we need the memory region information from ELF headers. So far, we have only run on binaries compiled with zero optimization.

References

[1] P. E. Metzger, "Slow but steady: Achieving real security within two decades," Keynote, Language-Theoretic Security Workshop (LangSec), IEEE Security and Privacy Workshops, 2017.

[2] L. Sassaman, M. L. Patterson, and S. Bratus, "Security applications of formal language theory," *IEEE Security & Privacy*, vol. 11, no. 3, pp. 52–59, 2013.

[3] T. A. Henzinger, R. Majumdar, and J.-F. Raskin, "A classification of symbolic transition systems," in *Proceedings of STACS 2000*, ser. Lecture Notes in Computer Science, vol. 1770. Springer, 2000, pp. 13–34.

[4] R. Paige and R. E. Tarjan, "Three partition refinement algorithms," *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.

[5] H. Jordan, B. Scholz, and P. Subotić, "Soufflé: On synthesis of program analyzers," in *Computer Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 9780. Springer, 2016, pp. 422–430.

[6] T. A. Henzinger, R. Jhala, R. Majumdar, and G. Sutre, "Lazy abstraction," in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2002, pp. 58–70.

[7] R. Alur and P. Madhusudan, "Visibly pushdown languages," in *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 2004, pp. 202–211.

[8] R. Milner, *Communication and Concurrency*. Prentice Hall, 1989.

[9] R. J. van Glabbeek, "The linear time – branching time spectrum," in *CONCUR '90: Theories of Concurrency: Unification and Extension*, ser. Lecture Notes in Computer Science, vol. 458. Springer, 1990, pp. 278–297.

[10] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, "cvc5: A versatile and industrial-strength SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 415–442.

[11] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SOK: (state of) the art of war: Offensive techniques in binary analysis,"

in *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.

[12] E. Voogd, Å. A. A. Kløvstad, E. B. Johnsen, and A. Wąsowski, "Compositional symbolic execution semantics," *Theoretical Computer Science*, vol. 1044, p. 115263, 2025.

[13] F. Bourdoncle, "Efficient chaotic iteration strategies with widenings," in *Formal Methods in Programming and Their Applications*, ser. Lecture Notes in Computer Science, vol. 735. Springer, 1993, pp. 128–141.

[14] S. H. Park, A. Lööw, S. Élie Ayoun, and P. Gardner, "Filtered simulation for binary lifting," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2025.